

C Program: Create and Display a Singly Linked List

```
#include <stdio.h>
#include <stdlib.h>

// Define structure for Node
struct Node {
    int data;
    struct Node* next;
};

// Function to create a new node
struct Node* createNode(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->data = value;
    newNode->next = NULL;
    return newNode;
}

// Function to insert node at the end
struct Node* insertAtEnd(struct Node* head, int value) {
    struct Node* newNode = createNode(value);

    if (head == NULL) {
        return newNode;
    }

    struct Node* temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }

    temp->next = newNode;
    return head;
}

// Function to display the linked list
void displayList(struct Node* head) {
    struct Node* temp = head;

    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
    }
}
```

```

        temp = temp->next;
    }
    printf("NULL\n");
}

// Main function
int main() {
    struct Node* head = NULL;
    int n, value;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter data for node %d: ", i + 1);
        scanf("%d", &value);
        head = insertAtEnd(head, value);
    }

    displayList(head);

    return 0;
}

```

C Program: Insert at Beginning in Singly Linked List

```

#include <stdio.h>
#include <stdlib.h>

// Define structure for Node
struct Node {
    int data;
    struct Node* next;
};

// Function to insert at beginning
struct Node* insertAtBeginning(struct Node* head, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->data = value;
    newNode->next = head; // Point new node to current head
    head = newNode;      // Make new node as head

    return head;
}

// Function to display the list
void displayList(struct Node* head) {
    struct Node* temp = head;

```

```

    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

// Main function
int main() {
    struct Node* head = NULL;
    int n, value;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter data to insert at beginning: ");
        scanf("%d", &value);
        head = insertAtBeginning(head, value);
    }

    displayList(head);

    return 0;
}

```

C Program: Insert at End in Singly Linked List

```

#include <stdio.h>
#include <stdlib.h>

// Define structure for Node
struct Node {
    int data;
    struct Node* next;
};

// Function to insert at end
struct Node* insertAtEnd(struct Node* head, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->data = value;

```

```

newNode->next = NULL;

// If list is empty
if (head == NULL) {
    return newNode;
}

// Traverse to last node
struct Node* temp = head;
while (temp->next != NULL) {
    temp = temp->next;
}

temp->next = newNode;
return head;
}

// Function to display list
void displayList(struct Node* head) {
    struct Node* temp = head;

    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

// Main function
int main() {
    struct Node* head = NULL;
    int n, value;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter data to insert at end: ");
        scanf("%d", &value);
        head = insertAtEnd(head, value);
    }

    displayList(head);

    return 0;
}

```

C Program: Insert at Any Position

```
#include <stdio.h>
#include <stdlib.h>

// Define structure
struct Node {
    int data;
    struct Node* next;
};

// Function to insert at any position
struct Node* insertAtPosition(struct Node* head, int value, int position) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->data = value;
    newNode->next = NULL;

    // Insert at beginning
    if (position == 1) {
        newNode->next = head;
        return newNode;
    }

    struct Node* temp = head;

    // Traverse to (position - 1)
    for (int i = 1; i < position - 1 && temp != NULL; i++) {
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("Invalid Position!\n");
        free(newNode);
        return head;
    }

    newNode->next = temp->next;
    temp->next = newNode;

    return head;
}

// Function to display list
void displayList(struct Node* head) {
    struct Node* temp = head;

    if (head == NULL) {
        printf("List is empty.\n");
    }
}
```

```

        return;
    }

    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

// Main function
int main() {
    struct Node* head = NULL;
    int n, value, position;

    printf("Enter number of initial nodes: ");
    scanf("%d", &n);

    // Create initial list by inserting
    for (int i = 0; i < n; i++) {
        struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
        scanf("%d", &value);
        newNode->data = value;
        newNode->next = head;
        head = newNode;
    }

    printf("Enter value to insert: ");
    scanf("%d", &value);

    printf("Enter position: ");
    scanf("%d", &position);

    head = insertAtPosition(head, value, position);

    displayList(head);

    return 0;
}

```

C Program: Delete Front Node from Singly Linked List

```

#include <stdio.h>
#include <stdlib.h>

// Define structure
struct Node {
    int data;
    struct Node* next;
};

// Function to insert at end (for testing)
struct Node* insertAtEnd(struct Node* head, int value) {

```

```

struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

if (newNode == NULL) {
    printf("Memory allocation failed!\n");
    exit(1);
}

newNode->data = value;
newNode->next = NULL;

if (head == NULL)
    return newNode;

struct Node* temp = head;
while (temp->next != NULL)
    temp = temp->next;

temp->next = newNode;
return head;
}

// Function to delete front node
struct Node* deleteFront(struct Node* head)
{
    if (head == NULL) {
        printf("List is empty! Cannot delete.\n");
        return NULL;
    }

    struct Node* temp = head;
    printf("Deleted element: %d\n", temp->data);

    head = head->next; // Move head to next node
    free(temp);       // Free old first node

    return head;
}

// Function to display list
void display(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    struct Node* temp = head;
    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

```

```

// Main function
int main() {
    struct Node* head = NULL;
    int n, value;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter data: ");
        scanf("%d", &value);
        head = insertAtEnd(head, value);
    }

    display(head);

    head = deleteFront(head);

    display(head);

    return 0;
}

```

C Program: Delete Last Node from Singly Linked List

```

#include <stdio.h>
#include <stdlib.h>

// Define structure
struct Node {
    int data;
    struct Node* next;
};

// Function to insert at end (for testing)
struct Node* insertAtEnd(struct Node* head, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->data = value;
    newNode->next = NULL;

    if (head == NULL)
        return newNode;

    struct Node* temp = head;
    while (temp->next != NULL)
        temp = temp->next;

```



```

    temp->next = newNode;
    return head;
}

// Function to delete last node
struct Node* deleteLast(struct Node* head) {
    if (head == NULL) {
        printf("List is empty! Cannot delete.\n");
        return NULL;
    }

    // If only one node
    if (head->next == NULL) {
        printf("Deleted element: %d\n", head->data);
        free(head);
        return NULL;
    }

    struct Node* temp = head;
    struct Node* prev = NULL;

    // Traverse to last node
    while (temp->next != NULL) {
        prev = temp;
        temp = temp->next;
    }

    printf("Deleted element: %d\n", temp->data);
    prev->next = NULL;
    free(temp);

    return head;
}

// Function to display list
void display(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    struct Node* temp = head;
    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

// Main function
int main() {
    struct Node* head = NULL;

```

```

int n, value;

printf("Enter number of nodes: ");
scanf("%d", &n);

for (int i = 0; i < n; i++) {
    printf("Enter data: ");
    scanf("%d", &value);
    head = insertAtEnd(head, value);
}

display(head);

head = deleteLast(head);

display(head);

return 0;
}
C Program: Delete Node at Any Position from Singly Linked List
#include <stdio.h>
#include <stdlib.h>

// Define structure
struct Node {
    int data;
    struct Node* next;
};

// Function to insert at end (for testing)
struct Node* insertAtEnd(struct Node* head, int value)
{
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL)
    {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->data = value;
    newNode->next = NULL;

    if (head == NULL)
        return newNode;

    struct Node* temp = head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    return head;
}

```

```

// Function to delete node at given position
struct Node* deleteAtPosition(struct Node* head, int position) {
    if (head == NULL) {
        printf("List is empty! Cannot delete.\n");
        return NULL;
    }

    struct Node* temp = head;

    // Delete first node
    if (position == 1) {
        printf("Deleted element: %d\n", temp->data);
        head = temp->next;
        free(temp);
        return head;
    }

    struct Node* prev = NULL;

    // Traverse to desired position
    for (int i = 1; i < position && temp != NULL; i++) {
        prev = temp;
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("Invalid position!\n");
        return head;
    }

    printf("Deleted element: %d\n", temp->data);
    prev->next = temp->next;
    free(temp);

    return head;
}

// Function to display list
void display(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    struct Node* temp = head;
    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

```

```

// Main function
int main() {
    struct Node* head = NULL;
    int n, value, position;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
    {
        printf("Enter data: ");
        scanf("%d", &value);
        head = insertAtEnd(head, value);
    }

    display(head);

    printf("Enter position to delete: ");
    scanf("%d", &position);

    head = deleteAtPosition(head, position);

    display(head);

    return 0;
}

```

C Program: Search an Element in Singly Linked List

```

#include <stdio.h>
#include <stdlib.h>

// Define structure
struct Node {
    int data;
    struct Node* next;
};

// Function to insert at end (for testing)
struct Node* insertAtEnd(struct Node* head, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->data = value;
    newNode->next = NULL;

    if (head == NULL)
        return newNode;
}

```

```

    struct Node* temp = head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    return head;
}

// Function to search element
void search(struct Node* head, int key) {
    struct Node* temp = head;
    int position = 1;
    int found = 0;

    while (temp != NULL) {
        if (temp->data == key) {
            printf("Element %d found at position %d\n", key, position);
            found = 1;
            break;
        }
        temp = temp->next;
        position++;
    }

    if (!found) {
        printf("Element %d not found in the list.\n", key);
    }
}

// Function to display list
void display(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    struct Node* temp = head;
    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

// Main function
int main() {
    struct Node* head = NULL;
    int n, value, key;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

```

```

    for (int i = 0; i < n; i++) {
        printf("Enter data: ");
        scanf("%d", &value);
        head = insertAtEnd(head, value);
    }

    display(head);

    printf("Enter element to search: ");
    scanf("%d", &key);

    search(head, key);

    return 0;
}

C Program: Print All Elements in Singly Linked List
#include <stdio.h>
#include <stdlib.h>

// Define structure
struct Node {
    int data;
    struct Node* next;
};

// Function to insert at end (for creating list)
struct Node* insertAtEnd(struct Node* head, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->data = value;
    newNode->next = NULL;

    if (head == NULL)
        return newNode;

    struct Node* temp = head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    return head;
}

// Function to print all elements
void printList(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
    }
}

```

```

        return;
    }

    struct Node* temp = head;

    printf("Elements in the Linked List: ");
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

// Main function
int main() {
    struct Node* head = NULL;
    int n, value;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter data: ");
        scanf("%d", &value);
        head = insertAtEnd(head, value);
    }

    printList(head);

    return 0;
}

```

C Program: Create and Display a doubly Linked List

```

#include <stdio.h>
#include <stdlib.h>

// Define structure for Doubly Linked List Node
struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};

// Function to insert at end
struct Node* insertAtEnd(struct Node* head, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->data = value;

```

```

newNode->prev = NULL;
newNode->next = NULL;

// If list is empty
if (head == NULL) {
    return newNode;
}

struct Node* temp = head;

// Traverse to last node
while (temp->next != NULL)
{
    temp = temp->next;
}

temp->next = newNode;
newNode->prev = temp;

return head;
}

// Function to display list (Forward)
void displayForward(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    struct Node* temp = head;
    printf("Doubly Linked List (Forward): ");
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

// Function to display list (Backward)
void displayBackward(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    struct Node* temp = head;

    // Move to last node
    while (temp->next != NULL) {
        temp = temp->next;
    }

    printf("Doubly Linked List (Backward): ");

```



```

while (temp != NULL) {
    printf("%d <-> ", temp->data);
    temp = temp->prev;
}
printf("NULL\n");
}

// Main function
int main() {
    struct Node* head = NULL;
    int n, value;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter data: ");
        scanf("%d", &value);
        head = insertAtEnd(head, value);
    }

    displayForward(head);
    displayBackward(head);

    return 0;
}

```

C Program: Insert at Beginning in doubly Linked List

```

#include <stdio.h>
#include <stdlib.h>

// Define structure for Doubly Linked List Node
struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};

// Function to insert at beginning
struct Node* insertAtBeginning(struct Node* head, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->data = value;
    newNode->prev = NULL;
    newNode->next = head;

    if (head != NULL) {
        head->prev = newNode;
    }
}

```

```

    }

    head = newNode;
    return head;
}

// Function to display forward
void displayForward(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    struct Node* temp = head;
    printf("Doubly Linked List (Forward): ");
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

// Main function
int main() {
    struct Node* head = NULL;
    int n, value;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter data to insert at beginning: ");
        scanf("%d", &value);
        head = insertAtBeginning(head, value);
    }

    displayForward(head);

    return 0;
}

```

C Program: Insert at End in doubly Linked List

```

#include <stdio.h>
#include <stdlib.h>

// Define structure for Doubly Linked List Node
struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};

// Function to insert at end

```

```

struct Node* insertAtEnd(struct Node* head, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->data = value;
    newNode->next = NULL;
    newNode->prev = NULL;

    // If list is empty
    if (head == NULL) {
        return newNode;
    }

    struct Node* temp = head;

    // Traverse to last node
    while (temp->next != NULL) {
        temp = temp->next;
    }

    temp->next = newNode;
    newNode->prev = temp;

    return head;
}

// Function to display list (Forward)
void display(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    struct Node* temp = head;
    printf("Doubly Linked List: ");
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

// Main function
int main() {
    struct Node* head = NULL;
    int n, value;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

```

```

    for (int i = 0; i < n; i++) {
        printf("Enter data: ");
        scanf("%d", &value);
        head = insertAtEnd(head, value);
    }

    display(head);

    return 0;
}

```

C Program: Insert at Any Position in doubly linked list

```

#include <stdio.h>
#include <stdlib.h>

// Define structure
struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};

// Function to insert at end (for initial creation)
struct Node* insertAtEnd(struct Node* head, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->data = value;
    newNode->next = NULL;
    newNode->prev = NULL;

    if (head == NULL)
        return newNode;

    struct Node* temp = head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    newNode->prev = temp;

    return head;
}

// Function to insert at any position
struct Node* insertAtPosition(struct Node* head, int value, int position) {

    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

```

```

if (newNode == NULL) {
    printf("Memory allocation failed!\n");
    exit(1);
}

newNode->data = value;

// Insert at beginning
if (position == 1) {
    newNode->prev = NULL;
    newNode->next = head;

    if (head != NULL)
        head->prev = newNode;

    return newNode;
}

struct Node* temp = head;

// Traverse to (position - 1)
for (int i = 1; i < position - 1 && temp != NULL; i++) {
    temp = temp->next;
}

if (temp == NULL) {
    printf("Invalid Position!\n");
    free(newNode);
    return head;
}

newNode->next = temp->next;
newNode->prev = temp;

if (temp->next != NULL)
    temp->next->prev = newNode;

temp->next = newNode;

return head;
}

// Function to display list
void display(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    struct Node* temp = head;
    printf("Doubly Linked List: ");
    while (temp != NULL) {

```

```

        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

// Main function
int main() {
    struct Node* head = NULL;
    int n, value, position;

    printf("Enter number of initial nodes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter data: ");
        scanf("%d", &value);
        head = insertAtEnd(head, value);
    }

    display(head);

    printf("Enter value to insert: ");
    scanf("%d", &value);

    printf("Enter position: ");
    scanf("%d", &position);

    head = insertAtPosition(head, value, position);

    display(head);

    return 0;
}

```

C Program: Delete Front Node from doubly Linked List

```

#include <stdio.h>
#include <stdlib.h>

// Define structure
struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};

// Function to insert at end (for testing)
struct Node* insertAtEnd(struct Node* head, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }
}

```

```

    }

    newNode->data = value;
    newNode->next = NULL;
    newNode->prev = NULL;

    if (head == NULL)
        return newNode;

    struct Node* temp = head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    newNode->prev = temp;

    return head;
}

// Function to delete front node
struct Node* deleteFront(struct Node* head)
{
    if (head == NULL)
    {
        printf("List is empty! Cannot delete.\n");
        return NULL;
    }

    struct Node* temp = head;
    printf("Deleted element: %d\n", temp->data);

    head = head->next; // Move head to next node

    if (head != NULL)
        head->prev = NULL;

    free(temp);
    return head;
}

// Function to display list
void display(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    struct Node* temp = head;
    printf("Doubly Linked List: ");
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
}

```

```

    printf("NULL\n");
}

// Main function
int main() {
    struct Node* head = NULL;
    int n, value;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter data: ");
        scanf("%d", &value);
        head = insertAtEnd(head, value);
    }

    display(head);

    head = deleteFront(head);

    display(head);

    return 0;
}

```

C Program: Delete Last Node from doubly Linked List

```

#include <stdio.h>
#include <stdlib.h>

// Define structure
struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};

// Function to insert at end (for testing)
struct Node* insertAtEnd(struct Node* head, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->data = value;
    newNode->next = NULL;
    newNode->prev = NULL;

    if (head == NULL)
        return newNode;
}

```



```

    struct Node* temp = head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    newNode->prev = temp;

    return head;
}

// Function to delete last node
struct Node* deleteLast(struct Node* head)
{
    if (head == NULL)
    {
        printf("List is empty! Cannot delete.\n");
        return NULL;
    }

    struct Node* temp = head;

    // If only one node
    if (temp->next == NULL) {
        printf("Deleted element: %d\n", temp->data);
        free(temp);
        return NULL;
    }

    // Traverse to last node
    while (temp->next != NULL)
        temp = temp->next;

    printf("Deleted element: %d\n", temp->data);
    temp->prev->next = NULL;          // Remove link
    free(temp);

    return head;
}

// Function to display list
void display(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    struct Node* temp = head;
    printf("Doubly Linked List: ");
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

```

```

}

// Main function
int main() {
    struct Node* head = NULL;
    int n, value;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter data: ");
        scanf("%d", &value);
        head = insertAtEnd(head, value);
    }

    display(head);

    head = deleteLast(head);

    display(head);

    return 0;
}

```

C Program: Delete Node at Any Position from doubly linked list

```

#include <stdio.h>
#include <stdlib.h>

// Define structure
struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};

// Function to insert at end (for creating list)
struct Node* insertAtEnd(struct Node* head, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->data = value;
    newNode->next = NULL;
    newNode->prev = NULL;

    if (head == NULL)
        return newNode;

    struct Node* temp = head;

```

```

while (temp->next != NULL)
    temp = temp->next;

temp->next = newNode;
newNode->prev = temp;

return head;
}

// Function to delete node at any position
struct Node* deleteAtPosition(struct Node* head, int position) {

    if (head == NULL) {
        printf("List is empty! Cannot delete.\n");
        return NULL;
    }

    struct Node* temp = head;

    // Delete first node
    if (position == 1) {
        printf("Deleted element: %d\n", temp->data);
        head = temp->next;

        if (head != NULL)
            head->prev = NULL;

        free(temp);
        return head;
    }

    // Traverse to desired position
    for (int i = 1; i < position && temp != NULL; i++) {
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("Invalid position!\n");
        return head;
    }

    printf("Deleted element: %d\n", temp->data);

    if (temp->next != NULL)
        temp->next->prev = temp->prev;

    if (temp->prev != NULL)
        temp->prev->next = temp->next;

    free(temp);

    return head;
}

```

```

// Function to display list
void display(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    struct Node* temp = head;
    printf("Doubly Linked List: ");
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

// Main function
int main() {
    struct Node* head = NULL;
    int n, value, position;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter data: ");
        scanf("%d", &value);
        head = insertAtEnd(head, value);
    }

    display(head);

    printf("Enter position to delete: ");
    scanf("%d", &position);

    head = deleteAtPosition(head, position);

    display(head);

    return 0;
}

```

C Program: Search an Element in doubly Linked List

```

#include <stdio.h>
#include <stdlib.h>

```

```

// Define structure
struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};

```

```

// Function to insert at end (for creating list)
struct Node* insertAtEnd(struct Node* head, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->data = value;
    newNode->next = NULL;
    newNode->prev = NULL;

    if (head == NULL)
        return newNode;

    struct Node* temp = head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    newNode->prev = temp;

    return head;
}

// Function to search element
void search(struct Node* head, int key) {
    struct Node* temp = head;
    int position = 1;
    int found = 0;

    while (temp != NULL) {
        if (temp->data == key) {
            printf("Element %d found at position %d\n", key, position);
            found = 1;
            break;
        }
        temp = temp->next;
        position++;
    }

    if (!found) {
        printf("Element %d not found in the list.\n", key);
    }
}

// Function to display list
void display(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }
}

```

```

    }

    struct Node* temp = head;
    printf("Doubly Linked List: ");
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

// Main function
int main() {
    struct Node* head = NULL;
    int n, value, key;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter data: ");
        scanf("%d", &value);
        head = insertAtEnd(head, value);
    }

    display(head);

    printf("Enter element to search: ");
    scanf("%d", &key);

    search(head, key);

    return 0;
}

```

C Program: Print All Elements in doubly Linked List

```

#include <stdio.h>
#include <stdlib.h>

// Define structure
struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};

// Function to insert at end (for creating list)
struct Node* insertAtEnd(struct Node* head, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }
}

```

```

    }

    newNode->data = value;
    newNode->next = NULL;
    newNode->prev = NULL;

    if (head == NULL)
        return newNode;

    struct Node* temp = head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    newNode->prev = temp;

    return head;
}

// Function to print all elements (Forward Traversal)
void displayForward(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    struct Node* temp = head;
    printf("Doubly Linked List (Forward): ");
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

// Function to print all elements (Backward Traversal)
void displayBackward(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    struct Node* temp = head;

    // Move to last node
    while (temp->next != NULL)
        temp = temp->next;

    printf("Doubly Linked List (Backward): ");
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->prev;
    }
}

```

```

    printf("NULL\n");
}

// Main function
int main() {
    struct Node* head = NULL;
    int n, value;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter data: ");
        scanf("%d", &value);
        head = insertAtEnd(head, value);
    }

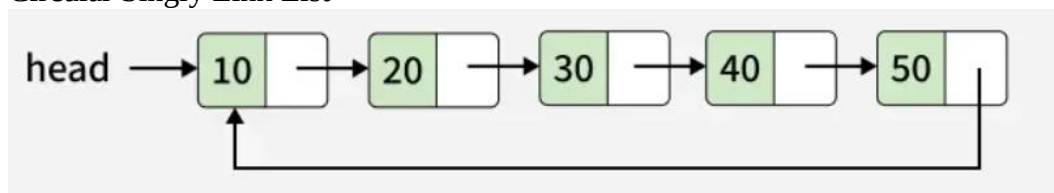
    displayForward(head);
    displayBackward(head);

    return 0;
}

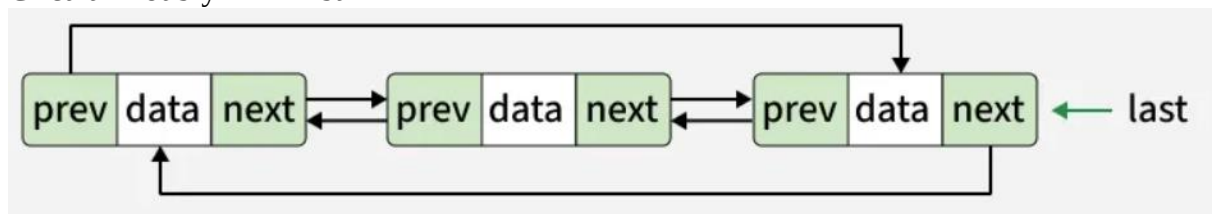
```

Circular Link List:

1. Circular Singly Link List



2. Circular Doubly Link List



Advantage of Circular Linked List

- Efficient Traversal
- No Null Pointers / References
- Useful for Repetitive Tasks
- Uniform Traversal
- Efficient Memory Utilization

Disadvantage of Circular Linked List

- Complex Implementation
- Infinite Loop Risk
- Harder to Debug
- Deletion Complexity

Applications:

1. Round Robin Scheduling
2. Multi-Player Games
3. Implementation of Circular Queues
4. Real-time Systems Like, Traffic Light Control
5. Image/Slide Show Viewers
6. Token Passing in Networks (Ring Topology)

C Program to Create and Display a Circular Singly Linked List

```
#include <stdio.h>
#include <stdlib.h>

// Structure definition
struct node
{
    int data;
    struct node *next;
};

struct node *head = NULL, *temp, *newnode;

// Function to create circular linked list
void create()
{
    int n, i;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for(i = 1; i <= n; i++)
    {
        newnode = (struct node*)malloc(sizeof(struct node));

        printf("Enter data for node %d: ", i);
        scanf("%d", &newnode->data);

        if(head == NULL)
        {
            head = newnode;
            temp = newnode;
            newnode->next = head; // Circular link
        }
        else
        {
            temp->next = newnode;
            newnode->next = head; // Circular link
            temp = newnode;
        }
    }
}

// Function to display circular linked list
void display()
{

```

```

struct node *ptr;

if(head == NULL)
{
printf("List is empty\n");
return;
}

ptr = head;

printf("Circular Linked List:\n");

do
{
printf("%d -> ", ptr->data);
ptr = ptr->next;
} while(ptr != head);

printf("(Back to Head)\n");
}

// Main Function
int main()
{
create();
display();

return 0;
}

```

C Program: Insert at Beginning in a Circular Singly Linked List

```

#include <stdio.h>
#include <stdlib.h>

// Structure definition
struct node
{
int data;
struct node *next;
};

struct node *head = NULL;

// Function to insert at beginning
void insert_beginning(int value)
{
struct node *newnode, *temp;

newnode = (struct node*)malloc(sizeof(struct node));
newnode->data = value;

// If list is empty
if(head == NULL)
{
head = newnode;

```

```

newnode->next = head;
}
else
{
temp = head;

// Move to last node
while(temp->next != head)
temp = temp->next;

newnode->next = head;
temp->next = newnode;
head = newnode;
}
}

// Function to display list
void display()
{
struct node *temp;

if(head == NULL)
{
printf("List is empty\n");
return;
}

temp = head;

printf("Circular Linked List:\n");

do
{
printf("%d -> ", temp->data);
temp = temp->next;
}
while(temp != head);

printf("HEAD\n");
}

// Main function
int main()
{
int n, i, value;

printf("Enter number of nodes: ");
scanf("%d", &n);

for(i = 1; i <= n; i++)
{
printf("Enter value %d: ", i);
scanf("%d", &value);

insert_beginning(value);

```

```
}
```

```
display();
```

```
return 0;
```

```
}
```

C Program: Insert at End in a Circular Singly Linked List

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Structure definition
```

```
struct node
```

```
{
```

```
int data;
```

```
struct node *next;
```

```
};
```

```
struct node *head = NULL;
```

```
// Function to insert at end
```

```
void insert_end(int value)
```

```
{
```

```
struct node *newnode, *temp;
```

```
newnode = (struct node*)malloc(sizeof(struct node));
```

```
newnode->data = value;
```

```
// If list is empty
```

```
if(head == NULL)
```

```
{
```

```
head = newnode;
```

```
newnode->next = head;
```

```
}
```

```
else
```

```
{
```

```
temp = head;
```

```
// Move to last node
```

```
while(temp->next != head)
```

```
{
```

```
temp = temp->next;
```

```
}
```

```
temp->next = newnode;
```

```
newnode->next = head;
```

```
}
```

```
}
```

```
// Function to display list
```

```
void display()
```

```
{
```

```
struct node *temp;
```

```
if(head == NULL)
```

```

{
printf("List is empty\n");
return;
}

temp = head;

printf("Circular Linked List:\n");

do
{
printf("%d -> ", temp->data);
temp = temp->next;
}
while(temp != head);

printf("HEAD\n");
}

// Main function
int main()
{
int n, i, value;

printf("Enter number of nodes: ");
scanf("%d", &n);

for(i = 1; i <= n; i++)
{
printf("Enter value %d: ", i);
scanf("%d", &value);

insert_end(value);
}

display();

return 0;
}

```

C Program: Insert at Any position in a Circular Singly Linked List

```

#include <stdio.h>
#include <stdlib.h>

// Structure definition
struct node
{
int data;
struct node *next;
};

struct node *head = NULL;

// Function to insert at any position
void insert_position(int value, int pos)

```

```

{
struct node *newnode, *temp;
int i;

newnode = (struct node*)malloc(sizeof(struct node));
newnode->data = value;

// Insert at beginning
if(pos == 1)
{
if(head == NULL)
{
head = newnode;
newnode->next = head;
}
else
{
temp = head;

while(temp->next != head)
temp = temp->next;

newnode->next = head;
temp->next = newnode;
head = newnode;
}
}
else
{
temp = head;

for(i = 1; i < pos-1; i++)
temp = temp->next;

newnode->next = temp->next;
temp->next = newnode;
}

}

// Function to display list
void display()
{
struct node *temp;

if(head == NULL)
{
printf("List is empty\n");
return;
}

temp = head;

printf("Circular Linked List:\n");

do

```

```

{
printf("%d -> ", temp->data);
temp = temp->next;
}
while(temp != head);

printf("HEAD\n");
}

// Main function
int main()
{
int n, i, value, pos;

printf("Enter number of nodes: ");
scanf("%d",&n);

for(i=1;i<=n;i++)
{
printf("Enter value: ");
scanf("%d",&value);

printf("Enter position: ");
scanf("%d",&pos);

insert_position(value,pos);
}

display();

return 0;
}

```

C Program: Delete a node at Beginning in a Circular Singly Linked List

```

#include <stdio.h>
#include <stdlib.h>

struct Node {
int data;
struct Node* next;
};

struct Node* head = NULL;

/* Create Circular Singly Linked List */
void create(int n) {
struct Node *temp, *newNode;
int data, i;

if (n <= 0) {
printf("Invalid number of nodes.\n");
return;
}

printf("Enter data for node 1: ");

```

```

scanf("%d", &data);

head = (struct Node*)malloc(sizeof(struct Node));
if (head == NULL) {
printf("Memory allocation failed.\n");
exit(0);
}

head->data = data;
head->next = head; // Circular link
temp = head;

for (i = 2; i <= n; i++) {
newNode = (struct Node*)malloc(sizeof(struct Node));
if (newNode == NULL) {
printf("Memory allocation failed.\n");
exit(0);
}

printf("Enter data for node %d: ", i);
scanf("%d", &data);

newNode->data = data;
newNode->next = head;
temp->next = newNode;
temp = newNode;
}

/* Delete Node at Beginning */
void deleteBeginning() {
struct Node *temp, *last;

// Condition 1: List Empty
if (head == NULL) {
printf("\nList is empty. Nothing to delete.\n");
return;
}

// Condition 2: Only One Node
if (head->next == head) {
temp = head;
head = NULL;
free(temp);
printf("\nOnly node deleted. List is now empty.\n");
return;
}

// Condition 3: Multiple Nodes
last = head;

// Traverse to last node
while (last->next != head) {

```



```

last = last->next;
}

temp = head; // Store old head
head = head->next; // Move head forward
last->next = head; // Maintain circular link

free(temp);

printf("\nFirst node deleted successfully.\n");
}

/* Display Circular List */
void display() {
struct Node *temp;

if (head == NULL) {
printf("List is empty.\n");
return;
}

temp = head;
printf("Circular Linked List: ");

do {
printf("%d -> ", temp->data);
temp = temp->next;
} while (temp != head);

printf("(Back to Head)\n");
}

/* Main Function */
int main() {
int n;

printf("Enter number of nodes: ");
scanf("%d", &n);

create(n);

printf("\nBefore deletion:\n");
display();

deleteBeginning();

printf("\nAfter deletion:\n");
display();

return 0;
}

```

C Program: Delete a node at End in a Circular Singly Linked List

```

#include <stdio.h>
#include <stdlib.h>

struct Node {
int data;
struct Node* next;
};

struct Node* head = NULL;

/* Create Circular Singly Linked List */
void create(int n) {
struct Node *temp, *newNode;
int data, i;

if (n <= 0) {
printf("Invalid number of nodes.\n");
return;
}

printf("Enter data for node 1: ");
scanf("%d", &data);

head = (struct Node*)malloc(sizeof(struct Node));
if (head == NULL) {
printf("Memory allocation failed.\n");
exit(0);
}

head->data = data;
head->next = head; // Circular link
temp = head;

for (i = 2; i <= n; i++) {
newNode = (struct Node*)malloc(sizeof(struct Node));
if (newNode == NULL) {
printf("Memory allocation failed.\n");
exit(0);
}

printf("Enter data for node %d: ", i);
scanf("%d", &data);

newNode->data = data;
newNode->next = head;
temp->next = newNode;
temp = newNode;
}

/* Delete Node at End */
void deleteEnd() {
struct Node *curr, *prev;

// Condition 1: Empty list

```

```

if (head == NULL) {
printf("\nList is empty. Nothing to delete.\n");
return;
}

// Condition 2: Only one node
if (head->next == head) {
free(head);
head = NULL;
printf("\nOnly node deleted. List is now empty.\n");
return;
}

// Condition 3: Multiple nodes
prev = NULL;
curr = head;

// Traverse to last node
while (curr->next != head) {
prev = curr;
curr = curr->next;
}

prev->next = head; // Maintain circular link
free(curr);

printf("\nLast node deleted successfully.\n");
}

/* Display Circular Linked List */
void display() {
struct Node *temp;

if (head == NULL) {
printf("List is empty.\n");
return;
}

temp = head;
printf("Circular Linked List: ");

do {
printf("%d -> ", temp->data);
temp = temp->next;
} while (temp != head);

printf("(Back to Head)\n");
}

/* Main Function */
int main() {
int n;

printf("Enter number of nodes: ");
scanf("%d", &n);

```

```

create(n);

printf("\nBefore deletion:\n");
display();

deleteEnd();

printf("\nAfter deletion:\n");
display();

return 0;
}

```

C Program: Delete a node at any position in a Circular Singly Linked List

```

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* head = NULL;

/* Create Circular Singly Linked List */
void create(int n) {
    struct Node *temp, *newNode;
    int data, i;

    if (n <= 0) {
        printf("Invalid number of nodes.\n");
        return;
    }

    printf("Enter data for node 1: ");
    scanf("%d", &data);

    head = (struct Node*)malloc(sizeof(struct Node));
    if (head == NULL) {
        printf("Memory allocation failed.\n");
        exit(0);
    }

    head->data = data;
    head->next = head;
    temp = head;

    for (i = 2; i <= n; i++) {
        newNode = (struct Node*)malloc(sizeof(struct Node));
        if (newNode == NULL) {
            printf("Memory allocation failed.\n");
            exit(0);
        }

        printf("Enter data for node %d: ", i);
    }

```

```

scanf("%d", &data);

newNode->data = data;
newNode->next = head;
temp->next = newNode;
temp = newNode;
}
}

/* Count Nodes */
int countNodes() {
int count = 0;
struct Node* temp = head;

if (head == NULL)
return 0;

do {
count++;
temp = temp->next;
} while (temp != head);

return count;
}

/* Delete Node at Any Position */
void deleteAtPosition(int pos) {
struct Node *temp, *prev, *last;
int count, i;

// Condition 1: Empty list
if (head == NULL) {
printf("\nList is empty. Nothing to delete.\n");
return;
}

count = countNodes();

// Condition 2: Invalid position
if (pos <= 0 || pos > count) {
printf("\nInvalid position.\n");
return;
}

// Condition 3: Delete first node
if (pos == 1) {

// Only one node
if (head->next == head) {
free(head);
head = NULL;
printf("\nOnly node deleted. List is now empty.\n");
return;
}

```

```

// More than one node
last = head;
while (last->next != head)
last = last->next;

temp = head;
head = head->next;
last->next = head;
free(temp);

printf("\nNode at position 1 deleted successfully.\n");
return;
}

// Condition 4 & 5: Delete middle or last node
prev = head;
for (i = 1; i < pos - 1; i++) {
prev = prev->next;
}

temp = prev->next;
prev->next = temp->next;

free(temp);

printf("\nNode at position %d deleted successfully.\n", pos);
}

/* Display Circular Linked List */
void display() {
struct Node *temp;

if (head == NULL) {
printf("List is empty.\n");
return;
}

temp = head;
printf("Circular Linked List: ");

do {
printf("%d -> ", temp->data);
temp = temp->next;
} while (temp != head);

printf("(Back to Head)\n");
}

/* Main Function */
int main() {
int n, pos;

printf("Enter number of nodes: ");
scanf("%d", &n);

```

```

create(n);

printf("\nBefore deletion:\n");
display();

printf("\nEnter position to delete: ");
scanf("%d", &pos);

deleteAtPosition(pos);

printf("\nAfter deletion:\n");
display();

return 0;
}

```

C Program: Search an Element in a Circular Singly Linked List

```

#include <stdio.h>
#include <stdlib.h>

struct Node {
int data;
struct Node* next;
};

struct Node* head = NULL;

/* Create Circular Singly Linked List */
void create(int n) {
struct Node *temp, *newNode;
int data, i;

if (n <= 0) {
printf("Invalid number of nodes.\n");
return;
}

printf("Enter data for node 1: ");
scanf("%d", &data);

head = (struct Node*)malloc(sizeof(struct Node));
if (head == NULL) {
printf("Memory allocation failed.\n");
exit(0);
}

head->data = data;
head->next = head; // Circular link
temp = head;

for (i = 2; i <= n; i++) {
newNode = (struct Node*)malloc(sizeof(struct Node));
if (newNode == NULL) {
printf("Memory allocation failed.\n");
exit(0);
}

```

```

}

printf("Enter data for node %d: ", i);
scanf("%d", &data);

newNode->data = data;
newNode->next = head;
temp->next = newNode;
temp = newNode;
}
}

/* Search Element */
void search(int key) {
    struct Node* temp;
    int position = 1;
    int found = 0;

    // Condition 1: Empty list
    if (head == NULL) {
        printf("\nList is empty.\n");
        return;
    }

    temp = head;

    // Traverse circular list
    do {
        if (temp->data == key) {
            printf("\nElement %d found at position %d.\n", key, position);
            found = 1;
            break;
        }
        temp = temp->next;
        position++;
    } while (temp != head);

    // Condition 5: Not found
    if (!found) {
        printf("\nElement %d not found in the list.\n", key);
    }
}

/* Display Circular Linked List */
void display() {
    struct Node *temp;

    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    temp = head;
    printf("Circular Linked List: ");

```



```

do {
printf("%d -> ", temp->data);
temp = temp->next;
} while (temp != head);

printf("(Back to Head)\n");
}

/* Main Function */
int main() {
int n, key;

printf("Enter number of nodes: ");
scanf("%d", &n);

create(n);

printf("\nList elements:\n");
display();

printf("\nEnter element to search: ");
scanf("%d", &key);

search(key);

return 0;
}

```

C Program: Insert at Beginning in a Circular Doubly Linked List

```

#include <stdio.h>
#include <stdlib.h>

struct Node {
int data;
struct Node* next;
struct Node* prev;
};

struct Node* head = NULL;

/* Insert at Beginning */
void insertBeginning(int data) {
struct Node* newNode;
struct Node* last;

// Allocate memory
newNode = (struct Node*)malloc(sizeof(struct Node));
if (newNode == NULL) {
printf("Memory allocation failed.\n");
return;
}

newNode->data = data;

// Condition 1: Empty list

```

```

if (head == NULL) {
    newNode->next = newNode;
    newNode->prev = newNode;
    head = newNode;
    printf("Node inserted as the only node in the list.\n");
    return;
}

// Condition 2: One or more nodes
last = head->prev; // Last node

newNode->next = head;
newNode->prev = last;

head->prev = newNode;
last->next = newNode;

head = newNode;

printf("Node inserted at the beginning successfully.\n");
}

/* Display Forward */
void displayForward() {
    struct Node* temp;

    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    temp = head;
    printf("Forward: ");
    do {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    } while (temp != head);
    printf("(Back to Head)\n");
}

/* Display Backward */
void displayBackward() {
    struct Node* temp;

    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    temp = head->prev; // Start from last
    printf("Backward: ");
    do {
        printf("%d <-> ", temp->data);
        temp = temp->prev;
    } while (temp != head->prev);
}

```

```

printf("(Back to Last)\n");
}

/* Main Function */
int main() {
int n, data, i;

printf("Enter number of nodes to insert at beginning: ");
scanf("%d", &n);

for (i = 1; i <= n; i++) {
printf("Enter data for node %d: ", i);
scanf("%d", &data);
insertBeginning(data);
}

printf("\nAfter insertion:\n");
displayForward();
displayBackward();

return 0;
}

```

C Program: Insert at End in a Circular Doubly Linked List

```

#include <stdio.h>
#include <stdlib.h>

struct Node {
int data;
struct Node* next;
struct Node* prev;
};

struct Node* head = NULL;

/* Insert at End */
void insertEnd(int data) {
struct Node* newNode;
struct Node* last;

// Allocate memory
newNode = (struct Node*)malloc(sizeof(struct Node));
if (newNode == NULL) {
printf("Memory allocation failed.\n");
return;
}

newNode->data = data;

// Condition 1: Empty list
if (head == NULL) {
newNode->next = newNode;
newNode->prev = newNode;
head = newNode;
printf("Node inserted as the only node in the list.\n");
return;
}

```

```

}

// Condition 2: One or more nodes
last = head->prev; // Last node

newNode->next = head;
newNode->prev = last;

last->next = newNode;
head->prev = newNode;

printf("Node inserted at the end successfully.\n");
}

/* Display Forward */
void displayForward() {
    struct Node* temp;

    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    temp = head;
    printf("Forward: ");
    do {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    } while (temp != head);
    printf("(Back to Head)\n");
}

/* Display Backward */
void displayBackward() {
    struct Node* temp;

    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    temp = head->prev; // Start from last
    printf("Backward: ");
    do {
        printf("%d <-> ", temp->data);
        temp = temp->prev;
    } while (temp != head->prev);
    printf("(Back to Last)\n");
}

/* Main Function */
int main() {
    int n, data, i;

    printf("Enter number of nodes to insert at end: ");

```

```

scanf("%d", &n);

for (i = 1; i <= n; i++) {
    printf("Enter data for node %d: ", i);
    scanf("%d", &data);
    insertEnd(data);
}

printf("\nAfter insertion:\n");
displayForward();
displayBackward();

return 0;
}

```

C Program: Insert at Any position in a Circular Doubly Linked List

```

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
    struct Node* prev;
};

struct Node* head = NULL;

/* Count Nodes */
int countNodes() {
    int count = 0;
    struct Node* temp = head;

    if (head == NULL)
        return 0;

    do {
        count++;
        temp = temp->next;
    } while (temp != head);

    return count;
}

/* Insert at Any Position */
void insertAtPosition(int data, int pos) {
    struct Node *newNode, *temp, *last;
    int count, i;

    newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed.\n");
        return;
    }

    newNode->data = data;

```

```

count = countNodes();

// Condition 1: Empty list
if (head == NULL) {
    if (pos != 1) {
        printf("Invalid position. List is empty.\n");
        free(newNode);
        return;
    }
    newNode->next = newNode;
    newNode->prev = newNode;
    head = newNode;
    printf("Node inserted as the only node in the list.\n");
    return;
}

// Condition 2: Invalid position
if (pos <= 0 || pos > count + 1) {
    printf("Invalid position.\n");
    free(newNode);
    return;
}

// Condition 3: Insert at Beginning
if (pos == 1) {
    last = head->prev;

    newNode->next = head;
    newNode->prev = last;

    last->next = newNode;
    head->prev = newNode;

    head = newNode;

    printf("Node inserted at beginning successfully.\n");
    return;
}

// Condition 4: Insert at End
if (pos == count + 1) {
    last = head->prev;

    newNode->next = head;
    newNode->prev = last;

    last->next = newNode;
    head->prev = newNode;

    printf("Node inserted at end successfully.\n");
    return;
}

// Condition 5: Insert at Middle
temp = head;

```

```

for (i = 1; i < pos - 1; i++) {
temp = temp->next;
}

newNode->next = temp->next;
newNode->prev = temp;

temp->next->prev = newNode;
temp->next = newNode;

printf("Node inserted at position %d successfully.\n", pos);
}

/* Display Forward */
void displayForward() {
struct Node* temp;

if (head == NULL) {
printf("List is empty.\n");
return;
}

temp = head;
printf("Forward: ");
do {
printf("%d <-> ", temp->data);
temp = temp->next;
} while (temp != head);
printf("(Back to Head)\n");
}

/* Display Backward */
void displayBackward() {
struct Node* temp;

if (head == NULL) {
printf("List is empty.\n");
return;
}

temp = head->prev;
printf("Backward: ");
do {
printf("%d <-> ", temp->data);
temp = temp->prev;
} while (temp != head->prev);
printf("(Back to Last)\n");
}

/* Main Function */
int main() {
int n, data, pos, i;

printf("Enter number of initial nodes: ");
scanf("%d", &n);

```

```

for (i = 1; i <= n; i++) {
printf("Enter data for node %d (insert at end): ", i);
scanf("%d", &data);
insertAtPosition(data, i); // inserting sequentially
}

printf("\nCurrent List:\n");
displayForward();

printf("\nEnter data to insert: ");
scanf("%d", &data);

printf("Enter position to insert: ");
scanf("%d", &pos);

insertAtPosition(data, pos);

printf("\nUpdated List:\n");
displayForward();
displayBackward();

return 0;
}

```

C Program: Delete a node at Beginning in a Circular Doubly Linked List

```

#include <stdio.h>
#include <stdlib.h>

```

```

struct Node {
int data;
struct Node* next;
struct Node* prev;
};

```

```

struct Node* head = NULL;

```

```

/* Delete Node at Beginning */
void deleteBeginning() {
struct Node *temp, *last;

```

```

// Condition 1: Empty list
if (head == NULL) {
printf("List is empty. Nothing to delete.\n");
return;
}

```

```

// Condition 2: Only one node
if (head->next == head) {
free(head);
head = NULL;
printf("Only node deleted. List is now empty.\n");
return;
}

```

```

// Condition 3: Multiple nodes

```



```

temp = head;
last = head->prev;

head = head->next; // Move head forward
head->prev = last;
last->next = head;

free(temp);

printf("First node deleted successfully.\n");
}

/* Insert at End (Helper for Testing) */
void insertEnd(int data) {
    struct Node* newNode;
    struct Node* last;

    newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed.\n");
        return;
    }

    newNode->data = data;

    if (head == NULL) {
        newNode->next = newNode;
        newNode->prev = newNode;
        head = newNode;
        return;
    }

    last = head->prev;

    newNode->next = head;
    newNode->prev = last;

    last->next = newNode;
    head->prev = newNode;
}

/* Display Forward */
void displayForward() {
    struct Node* temp;

    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    temp = head;
    printf("Forward: ");
    do {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    } while (temp != head);
    printf("\n");
}

```

```

} while (temp != head);
printf("(Back to Head)\n");
}

/* Display Backward */
void displayBackward() {
    struct Node* temp;

    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    temp = head->prev;
    printf("Backward: ");
    do {
        printf("%d <-> ", temp->data);
        temp = temp->prev;
    } while (temp != head->prev);
    printf("(Back to Last)\n");
}

/* Main Function */
int main() {
    int n, data, i;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++) {
        printf("Enter data for node %d: ", i);
        scanf("%d", &data);
        insertEnd(data);
    }

    printf("\nBefore deletion:\n");
    displayForward();

    deleteBeginning();

    printf("\nAfter deletion:\n");
    displayForward();
    displayBackward();

    return 0;
}

```

C Program: Delete a node at End in a Circular Doubly Linked List

```

#include <stdio.h>
#include <stdlib.h>

```

```

struct Node {
    int data;
    struct Node* next;
    struct Node* prev;
};

```

```

struct Node* head = NULL;

/* Delete Node at End */
void deleteEnd() {
    struct Node *last, *secondLast;

    // Condition 1: Empty list
    if (head == NULL) {
        printf("List is empty. Nothing to delete.\n");
        return;
    }

    // Condition 2: Only one node
    if (head->next == head) {
        free(head);
        head = NULL;
        printf("Only node deleted. List is now empty.\n");
        return;
    }

    // Condition 3: Multiple nodes
    last = head->prev;
    secondLast = last->prev;

    secondLast->next = head;
    head->prev = secondLast;

    free(last);

    printf("Last node deleted successfully.\n");
}

/* Insert at End (Helper for Testing) */
void insertEnd(int data) {
    struct Node* newNode;
    struct Node* last;

    newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed.\n");
        return;
    }

    newNode->data = data;

    if (head == NULL) {
        newNode->next = newNode;
        newNode->prev = newNode;
        head = newNode;
        return;
    }

    last = head->prev;

```

```
newNode->next = head;
newNode->prev = last;
```

```
last->next = newNode;
head->prev = newNode;
}
```

```
/* Display Forward */
void displayForward() {
    struct Node* temp;
```

```
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }
```

```
    temp = head;
    printf("Forward: ");
    do {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    } while (temp != head);
    printf("(Back to Head)\n");
}
```

```
/* Display Backward */
void displayBackward() {
    struct Node* temp;
```

```
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }
```

```
    temp = head->prev;
    printf("Backward: ");
    do {
        printf("%d <-> ", temp->data);
        temp = temp->prev;
    } while (temp != head->prev);
    printf("(Back to Last)\n");
}
```

```
/* Main Function */
int main() {
    int n, data, i;
```

```
    printf("Enter number of nodes: ");
    scanf("%d", &n);
```

```
    for (i = 1; i <= n; i++) {
        printf("Enter data for node %d: ", i);
        scanf("%d", &data);
        insertEnd(data);
    }
```

```
printf("\nBefore deletion:\n");
displayForward();
```

```
deleteEnd();
```

```
printf("\nAfter deletion:\n");
displayForward();
displayBackward();
```

```
return 0;
}
```

C Program: Delete a node at any position in a Circular Doubly Linked List

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {
    int data;
    struct Node* next;
    struct Node* prev;
};
```

```
struct Node* head = NULL;
```

```
/* Count Nodes */
```

```
int countNodes() {
    int count = 0;
    struct Node* temp = head;
```

```
    if (head == NULL)
        return 0;
```

```
    do {
        count++;
        temp = temp->next;
    } while (temp != head);
```

```
    return count;
}
```

```
/* Delete at Any Position */
```

```
void deleteAtPosition(int pos) {
    struct Node *temp, *last;
    int count, i;
```

```
// Condition 1: Empty list
```

```
if (head == NULL) {
    printf("List is empty. Nothing to delete.\n");
    return;
}
```

```
count = countNodes();
```

```
// Condition 2: Invalid position
```

```
if (pos <= 0 || pos > count) {
```

```

printf("Invalid position.\n");
return;
}

// Condition 3: Only one node
if (head->next == head) {
free(head);
head = NULL;
printf("Only node deleted. List is now empty.\n");
return;
}

// Condition 4: Delete at Beginning
if (pos == 1) {
temp = head;
last = head->prev;

head = head->next;
head->prev = last;
last->next = head;

free(temp);
printf("Node at position 1 deleted successfully.\n");
return;
}

// Condition 5: Delete at End
if (pos == count) {
temp = head->prev; // Last node
temp->prev->next = head; // Second last next = head
head->prev = temp->prev; // Update head prev

free(temp);
printf("Last node deleted successfully.\n");
return;
}

// Condition 6: Delete at Middle
temp = head;
for (i = 1; i < pos; i++) {
temp = temp->next;
}

temp->prev->next = temp->next;
temp->next->prev = temp->prev;

free(temp);
printf("Node at position %d deleted successfully.\n", pos);
}

/* Insert at End (Helper for Testing) */
void insertEnd(int data) {
struct Node* newNode;
struct Node* last;

```

```
newNode = (struct Node*)malloc(sizeof(struct Node));
if (newNode == NULL) {
printf("Memory allocation failed.\n");
return;
}
```

```
newNode->data = data;
```

```
if (head == NULL) {
newNode->next = newNode;
newNode->prev = newNode;
head = newNode;
return;
}
```

```
last = head->prev;
```

```
newNode->next = head;
newNode->prev = last;
```

```
last->next = newNode;
head->prev = newNode;
}
```

```
/* Display Forward */
void displayForward() {
struct Node* temp;
```

```
if (head == NULL) {
printf("List is empty.\n");
return;
}
```

```
temp = head;
printf("Forward: ");
do {
printf("%d <-> ", temp->data);
temp = temp->next;
} while (temp != head);
printf("(Back to Head)\n");
}
```

```
/* Main Function */
int main() {
int n, data, pos, i;
```

```
printf("Enter number of nodes: ");
scanf("%d", &n);
```

```
for (i = 1; i <= n; i++) {
printf("Enter data for node %d: ", i);
scanf("%d", &data);
insertEnd(data);
}
```

```
printf("\nBefore deletion:\n");
displayForward();
```

```
printf("\nEnter position to delete: ");
scanf("%d", &pos);
```

```
deleteAtPosition(pos);
```

```
printf("\nAfter deletion:\n");
displayForward();
```

```
return 0;
}
```

C Program: Search an Element in a Circular Doubly Linked List

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {
    int data;
    struct Node* next;
    struct Node* prev;
};
```

```
struct Node* head = NULL;
```

```
/* Insert at End (Helper Function) */
```

```
void insertEnd(int data) {
    struct Node* newNode;
    struct Node* last;
```

```
    newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed.\n");
        return;
    }
```

```
    newNode->data = data;
```

```
// Empty list
```

```
if (head == NULL) {
    newNode->next = newNode;
    newNode->prev = newNode;
    head = newNode;
    return;
}
```

```
last = head->prev;
```

```
newNode->next = head;
newNode->prev = last;
```

```
last->next = newNode;
head->prev = newNode;
}
```



```

/* Search Element */
void search(int key) {
    struct Node* temp;
    int position = 1;
    int found = 0;

    // Condition 1: Empty list
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    temp = head;

    // Traverse circular doubly linked list
    do {
        if (temp->data == key) {
            printf("Element %d found at position %d.\n", key, position);
            found = 1;
            break;
        }
        temp = temp->next;
        position++;
    } while (temp != head);

    // Condition 5: Not found
    if (!found) {
        printf("Element %d not found in the list.\n", key);
    }
}

/* Display Forward */
void displayForward() {
    struct Node* temp;

    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    temp = head;
    printf("Forward: ");
    do {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    } while (temp != head);
    printf("(Back to Head)\n");
}

/* Display Backward */
void displayBackward() {
    struct Node* temp;

    if (head == NULL) {
        printf("List is empty.\n");
    }
}

```

```
return;  
}
```

```
temp = head->prev;  
printf("Backward: ");  
do {  
    printf("%d <-> ", temp->data);  
    temp = temp->prev;  
} while (temp != head->prev);  
printf("(Back to Last)\n");  
}
```

```
/* Main Function */
```

```
int main() {  
    int n, data, key, i;
```

```
    printf("Enter number of nodes: ");  
    scanf("%d", &n);
```

```
    for (i = 1; i <= n; i++) {  
        printf("Enter data for node %d: ", i);  
        scanf("%d", &data);  
        insertEnd(data);  
    }
```

```
    printf("\nList elements:\n");  
    displayForward();
```

```
    printf("\nEnter element to search: ");  
    scanf("%d", &key);
```

```
    search(key);
```

```
    return 0;  
}
```